
ARCHITECTURE BLUEPRINT · CMS POSTS/PAGES AND
WIDGET COMPOSITION

Necoyoad

v8

*How posts and pages share a table, how widgets compose
with them dynamically and manually, and how templates
can be overridden per entity*

scope: post table (shared by posts/pages), content controllers,
widgets-common.tpl, widgets-rows.tpl, per-entity template
override, only: prefix, page_embed.tpl, landing pages
findings: every CMS page is a widget composition surface,
dynamic vs manual vs hybrid composition, only: optimisation
listings: 12 PHP · **pages:** 14

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — CMS Posts/Pages and Widget Composition (v8)

How posts and pages share a table, how widgets compose with them dynamically and manually, and how templates can be overridden per entity

Abstract

This is the eighth volume of the Necoyoad architecture blueprint. v8 covers the CMS subsystem (posts and pages) and its deep integration with the widget composition system. The key finding is that posts and pages share the same database table (`post`) with a `post_type` discriminator, and that every CMS page is a widget composition surface — the same `loadWidgets()/addChild()/render()` pipeline that drives the home page and product pages also drives blog posts, static pages, and post categories.

v8 documents two composition modes: *dynamic composition* (the admin configures widgets via the visual editor, and the `widgets-rows.tpl` template emits `{%widget_name%}` placeholders that `Controller::fetch()` substitutes at render time) and *manual hardcoding* (a template author writes `{%widget_name%}` tokens directly in the `.tpl` file, bypassing the admin widget manager). It also covers the per-entity template override mechanism (EAV `property('style', 'view')` returns a template path), the `widgets-common.tpl` universal layout fragment, the `only:` position prefix for embedded pages, and the `page_embed.tpl` simplified layout used by the menu system's `submenu_type = 'page_id'`.

The stance is unchanged: **descriptive, not prescriptive**.

Keywords: Necoyoad, OpenCart, PHP 8.1, CMS, posts, pages, widget composition, dynamic rendering, hardcoded placeholders, template override, `widgets-common.tpl`, `widgets-rows.tpl`, `only:` prefix, `page_embed.tpl`, EAV property, polymorphic description.

Contents

1	Introduction	3
1.1	Eight Volumes	3
1.2	Scope	3
2	The Shared post Table	3
3	The Storefront CMS Controllers	4
3.1	The Common CMS Page Pattern	4
3.2	The Session <code>object_type</code> Protocol	5
3.3	Customer Group Permission Check	5
3.4	The Per-Entity Template Override	5
4	The <code>widgets-common.tpl</code> Universal Layout	6
4.1	Which Templates Use <code>widgets-common.tpl</code>	7
5	The <code>widgets-rows.tpl</code> Placeholder Emitter	7
5.1	The Row and Column Settings	8
6	Dynamic vs Manual Widget Composition	8
6.1	Dynamic Composition (The Default)	8
6.2	Manual Composition (Hardcoded Placeholders)	9
6.3	Hybrid Composition	9
7	The <code>only</code>: Position Prefix and Embedded Pages	10
7.1	The <code>page_embed.tpl</code> Layout	10
7.2	The <code>only</code> : Prefix	10
8	The CMS Lifecycle Trace	11
8.1	Visitor Views a Blog Post	11
8.2	Menu Embeds a Page	12
9	Cross-Cutting Findings	12
9.1	Every CMS Page is a Widget Composition Surface	12
9.2	The <code>widgets-common.tpl</code> Central Layout is a Refactoring	12
9.3	Manual Placeholders Require Admin Coordination	12
9.4	The <code>only</code> : Prefix is an Optimisation	12
9.5	Posts and Pages Share Everything But the Discriminator	13
10	Closing Observations	13
10.1	The CMS + Widget Integration is the Platform's Crown Jewel	13
10.2	The Manual Composition Mode is a Safety Valve	13
10.3	Final Note	13
A	Glossary	13

1 Introduction

1.1 Eight Volumes

v1–v7 covered the whole platform, source verification, the rendering pipeline, the marketing subsystem, multi-store/multi-language, the admin back-office, and the menu links system. v8 covers the CMS posts/pages subsystem and its integration with the widget composition system.

1.2 Scope

v8 covers:

- The `post` table — shared by posts and pages via `post_type`.
- The storefront content controllers (`content/post`, `content/page`, `content/category`).
- The per-entity template override mechanism (EAV `property('style', 'view')`).
- The `widgets-common.tpl` universal layout fragment.
- The `widgets-rows.tpl` placeholder emitter.
- Dynamic widget composition (admin-configured, via the visual editor).
- Manual widget composition (hardcoded `{%widget_name%}` tokens in `.tpl` files).
- The `only`: position prefix for embedded pages.
- The `page_embed.tpl` simplified layout.
- Landing page templates (combining widget-driven and hardcoded content).

2 The Shared post Table

Posts and pages share the same table

The `post` table stores both blog posts and static pages. The `post_type` column discriminates them: `'post'` for blog posts, `'page'` for static pages. Both the post model (`ModelContentPost`) and the page model (`ModelContentPage`) declare `$table = "post"` and `$pkey = "post_id"`. The only difference is `$object_type`: `'post'` vs `'page'`. This discriminator flows into the polymorphic description, property, and object_to_store tables.

```

1 CREATE TABLE 'post' (
2   'post_id'          int(11) NOT NULL,
3   'parent_id'        int(11) NOT NULL,          -- tree via parent_id
4   'author_id'        int(11) NOT NULL,          -- admin user who wrote it
5   'post_type'        varchar(50) NOT NULL,      -- 'post' or 'page'
6   'sort_order'       int(11) NOT NULL,
7   'image'            varchar(250) NOT NULL,
8   'status'           int(1) NOT NULL,
9   'date_publish_start' datetime NOT NULL,
10  'date_publish_end'  datetime NOT NULL,
11  'publish'           int(1) NOT NULL,          -- published flag
12  'allow_reviews'     int(1) NOT NULL,
13  'template'         varchar(100) NOT NULL,     -- template hint (unused by
14                        storefront)
15  'date_added'        datetime NOT NULL,
16  'date_modified'     datetime NOT NULL
17 );

```

Listing 1: The post table schema (shared by posts and pages).

The `template` column in the `post` table is declared but *not used* by the storefront controllers. The actual template override comes from the EAV property table, not from this column.

3 The Storefront CMS Controllers

3.1 The Common CMS Page Pattern

All three storefront CMS controllers (post, page, category) follow the same pattern:

```

1 public function index() {
2     $this->language->load('content/post');
3     $this->load->model('content/post');
4
5     // 1. Cache key (per-entity + language + currency + store + customer)
6     $cacheId = 'html-post.' . $this->request->getQuery('post_id') . ...;
7
8     // 2. Set session object_type/object_id (for per-object widget filtering)
9     $this->session->set('object_type', 'post');
10    $this->session->set('object_id', $post_id);
11
12    // 3. Load the entity
13    $post_info = $this->modelPost->getById($post_id);
14
15    if ($post_info && $post_info['publish']) {
16        // 4. Customer group permission check
17        $customerGroups = $this->modelPost->getProperty($post_id,
18            'customer_groups', 'customer_groups');
19        if (in_array(0, $customerGroups) ||
20            in_array($this->customer->getCustomerId(), $customerGroups))
21        {
22
23            // 5. Cache check
24            $cached = $this->cache->get($cacheId);
25            if ($cached && !$this->user->isLoggedIn()) {
26                $this->response->setOutput($cached, ...);
27            } else {
28                // 6. Set landing_page (for widget filtering)
29                $this->session->set('landing_page', 'content/post/index');
30
31                // 7. Load widgets for three positions
32                $this->loadWidgets('featuredContent');
33                $this->loadWidgets('main');
34                $this->loadWidgets('featuredFooter');
35
36                // 8. Add layout children
37                $this->addChild('common/column_left');
38                $this->addChild('common/column_right');
39                $this->addChild('common/header');
40                $this->addChild('common/footer');
41
42                // 9. Per-entity template override
43                $template = $this->modelPost->getProperty($post_id, 'style',
44                    'view');
45                $default_template = $this->config->get('default_view_post')
46                    ?: 'content/post.tpl';
47                $template = empty($template) ? $default_template : $template;
48                if (file_exists(DIR_TEMPLATE .
49                    $this->config->get('config_template') . '/' . $template))
50                {

```

```

44         $this->template = $this->config->get('config_template') .
           '/' . $template;
45     } else {
46         $this->template = 'choroni/' . $template;
47     }
48
49     // 10. Render (with cache write-back)
50     if (!$this->user->isLoggedIn()) $this->cacheId = $cacheId;
51     $this->response->setOutput($this->render(true), ...);
52 }
53 }
54 } else {
55     $this->error404();
56 }
57 }

```

Listing 2: The common CMS page pattern (abridged from `content/post.php`).

3.2 The Session `object_type` Protocol

Each CMS controller sets the session's `object_type` and `object_id`:

Table 1: Session `object_type` per CMS controller.

Controller	<code>object_type</code>	Effect
<code>content/post</code>	<code>'post'</code>	Widgets filtered by LIKE <code>'%object_type=post%'</code> and LIKE <code>'%object_id=<post_id>%'</code> .
<code>content/page</code>	<code>'page'</code>	Widgets filtered by LIKE <code>'%object_type=page%'</code> and LIKE <code>'%object_id=<page_id>%'</code> .
<code>content/category</code>	<code>'post_category'</code>	Widgets filtered by LIKE <code>'%object_type=post_category%'</code> and LIKE <code>'%object_id=<cat_id>%'</code> .

This is the mechanism v5 documented: `Controller::loadWidgets()` reads the session's `object_type/object_id` and runs a second `NecoWidget::getRows()` query with LIKE predicates, merging per-object widgets into the default tree.

3.3 Customer Group Permission Check

Every CMS entity has a `customer_groups` property in the EAV `property` table (under `group='customer_groups'`, `key='customer_groups'`). This is a serialised array of customer group IDs that are allowed to view the entity. If the array contains 0, the entity is public (visible to all). Otherwise, the customer's group must be in the array.

3.4 The Per-Entity Template Override

The template resolution follows a 3-level priority:

1. **EAV property** `property('style', 'view')` — the admin can set a per-entity template path (e.g. `content/landing_page_necotienda.tpl`) via the visual editor. This is stored in the property EAV table under `group='style'`, `key='view'`.
2. **Config default** — `default_view_post` or `default_view_page` setting (per-store, in the setting table). Falls back to `content/post.tpl` or `content/page.tpl`.
3. **Theme fallback** — if the template file exists in the active theme (`config_template`), use it; otherwise fall back to `choroni/`.

```

1 $template = $this->modelPost->getProperty($post_id, 'style', 'view');
2 $default_template = ($this->config->get('default_view_post'))
3   ? $this->config->get('default_view_post')
4   : 'content/post.tpl';
5 $template = empty($template) ? $default_template : $template;
6 if (file_exists(DIR_TEMPLATE . $this->config->get('config_template') . '/' .
7   $template)) {
8   $this->template = $this->config->get('config_template') . '/' . $template;
9 } else {
10  $this->template = 'choroni/' . $template;
11 }

```

Listing 3: Per-entity template override (from content/post.php).

This means each individual post or page can use a completely different template — a blog post can use `content/post.tpl`, a landing page can use `content/landing_page_necotienda.tpl`, and a product information page can use a custom `content/product-info.tpl` — all without changing any code.

4 The widgets-common.tpl Universal Layout

The most important template in the theme is `shared/widgets-common.tpl`. It is the universal layout fragment included by almost every page template:

```

1 <?php include("widgets-featured.tpl");?>
2
3 <!--mainContentContainer -->
4 <div id="mainContentContainer" nt-editable>
5   <div class="row">
6
7     <?php include("breadcrumbs.tpl");?>
8     <div class="clear"></div>
9
10    <!-- left-column -->
11    <?php if ($column_left) { ?>
12    <?php include("widgets-column-left.tpl");?>
13    <?php } ?>
14    <!--/left-column -->
15
16    <!--center-column -->
17    <?php include("widgets-column-center.tpl");?>
18    <!--/center-column -->
19
20    <!-- right-column -->
21    <?php if ($column_right) { ?>
22    <?php include("widgets-column-right.tpl");?>
23    <?php } ?>
24    <!--/right-column -->
25
26  </div>
27 </div>
28
29 <?php include("widgets-featured-footer.tpl");?>

```

Listing 4: `shared/widgets-common.tpl` — the universal layout fragment.

This template includes five fragments:

1. `widgets-featured.tpl` — sets `$position = 'featuredContent'` and includes `widgets-rows.tpl`.
2. `breadcrumbs.tpl` — renders the breadcrumb trail.

3. `widgets-column-left.tpl` — echoes `$column_left` (the rendered left column child).
4. `widgets-column-center.tpl` — sets `$position = 'main'` and includes `widgets-rows.tpl`. Also computes the column width based on which side columns are present.
5. `widgets-column-right.tpl` — echoes `$column_right`.
6. `widgets-featured-footer.tpl` — sets `$position = 'featuredFooter'` and includes `widgets-rows.tpl`.

4.1 Which Templates Use `widgets-common.tpl`

Table 2: Templates that include `widgets-common.tpl`.

Template	Controller	Notes
<code>content/post.tpl</code>	<code>content/post</code>	Blog post page.
<code>content/page.tpl</code>	<code>content/page</code>	Static page.
<code>store/product.tpl</code>	<code>store/product</code>	Product detail page.
<code>content/category.tpl</code>	<code>content/category</code>	Blog category page.
<code>content/categories.tpl</code>	<code>content/category/all</code>	All blog categories.
<code>content/posts.tpl</code>	<code>content/post/all</code>	All blog posts.

The `common/home.tpl` (home page) does *not* use `widgets-common.tpl` — it manually includes each fragment individually. This is an older pattern that predates `widgets-common.tpl`. The home page template also omits the breadcrumbs fragment.

5 The `widgets-rows.tpl` Placeholder Emitter

`shared/widgets-rows.tpl` is the template that actually emits the `{%widget_name%}` placeholder tokens. It iterates the `$rows[$position]` array (populated by `Controller::loadWidgets()`) and emits the widget tree as nested HTML with placeholder tokens:

```

1  <?php foreach($rows[$position] as $j => $row) { ?>
2      <?php if (!$row['key']) continue; ?>
3      <?php $row_id = $row['key']; ?>
4      <?php $row_settings = unserialize($row['value']); ?>
5
6      <div data-row="<?php echo $row_id; ?>" data-position="<?php echo
7          $position; ?>"
8          class="row<?php if (!empty($row_settings['classnames'])) echo '
9              '.$row_settings['classnames']; ?>"
10             id="<?php echo $position; ?>_<?php echo $row_id; ?>" nt-editable>
11
12         <?php foreach($row['columns'] as $k => $column) { ?>
13             <?php $column_id = $column['key']; ?>
14             <?php $column_settings = unserialize($column['value']); ?>
15
16             <div data-column="<?php echo $column_id; ?>" data-position="<?php
17                 echo $position; ?>"
18                 class="large-<?php echo $column_settings['grid_large']; ?>
19                     medium-<?php echo $column_settings['grid_medium']; ?>
20                     small-<?php echo $column_settings['grid_small']; ?>"
21                 id="<?php echo $position; ?>_<?php echo $column_id; ?>"
22                 nt-editable>
23                 <ul class="widgets">
24                     <?php foreach($column['widgets'] as $l => $widget) { ?>
25                         {%<?php echo $widget['name']; ?>%}
26                     <?php } ?>
27                 </ul>

```

```
24         </div>
25         <?php } ?>
26     </div>
27 <?php } ?>
```

Listing 5: shared/widgets-rows.tpl — the placeholder emitter.

The key line is: `<?php foreach($column['widgets'] as $widget) { ?> {%<?php echo $widget['name']; ?>%} <?php } ?>`

This emits `{%widget_name%}` for each widget in each column in each row. The `Controller::fetch()` method (v3, §3.2) then substitutes each `{%widget_name%}` token with the corresponding `$<name>_code` output (the widget's rendered HTML).

5.1 The Row and Column Settings

Each row and column has a serialised PHP settings blob (in the `value` column of the `property` EAV table). The `widgets-rows.tpl` template unserialises these and uses them for:

- **Row settings:** `sticky` (data-sticky attribute), `layout_width` ('fixed' adds the `container` class), `classnames` (additional CSS classes).
- **Column settings:** `grid_large`, `grid_medium`, `grid_small` (Foundation grid column widths for responsive layouts), `sticky`, `classnames`.

The `nt-editable` attribute on every row and column div is the visual theme editor's hook (v3, §9).

6 Dynamic vs Manual Widget Composition

Necoyoad supports two ways to compose widgets on a CMS page: *dynamic* (admin-configured via the visual editor) and *manual* (hardcoded `{%widget_name%}` tokens in the `.tpl` file).

6.1 Dynamic Composition (The Default)

In dynamic composition, the admin uses the visual theme editor to drag widgets into rows and columns. The widget tree is stored in the `property` EAV table (`object_type` = 'widget_rows', `group` = <position>). At render time:

1. `Controller::loadWidgets('main')` queries `NecoWidget::getRows()` which reads the `property` table.
2. The widget routes are merged into `$this->children` with string keys.
3. `Controller::render()` iterates `children`, rendering each widget's HTML into `$this->data[$name . '_code']`.
4. `Controller::fetch()` requires the template, which includes `widgets-rows.tpl`.
5. `widgets-rows.tpl` emits `{%widget_name%}` tokens.
6. `fetch()` substitutes the tokens with the widget HTML.

This is the default mode. The admin has full control over which widgets appear, in what order, in which columns, on which pages. No code changes are needed.

6.2 Manual Composition (Hardcoded Placeholders)

In manual composition, a template author writes `{%widget_name%}` tokens directly in the `.tpl` file, bypassing the admin widget manager. For this to work, the widget must still be registered as a child of the controller (via `loadWidgets()` or `addChild()`), so that its HTML is rendered into `$this->data[$name . '_code']`.

```

1 <!-- A custom landing page template that hardcodes widget positions -->
2 <div class="hero-section">
3     {%hero_banner_widget%}
4 </div>
5
6 <div class="features-grid">
7     <div class="col-4">{%product_overview_1%}</div>
8     <div class="col-4">{%product_overview_2%}</div>
9     <div class="col-4">{%product_overview_3%}</div>
10 </div>
11
12 <div class="content-section">
13     {%richtext_widget_1%}
14 </div>
15
16 <div class="cta-section">
17     {%contact_form_widget%}
18 </div>

```

Listing 6: Manual widget placeholder hardcoding in a custom template.

For this to work, the controller must still call `loadWidgets('main')` (or whatever position the widgets are assigned to), so that the widget children are registered and their HTML is rendered. The `widgets-rows.tpl` template is *not* included; instead, the custom template emits the placeholders directly in the desired HTML structure.

Manual placeholders require widget children to be registered

A `{%widget_name%}` token in a template is only substituted if the corresponding widget is registered as a child of the controller (in `$this->children` with the matching string key). This happens automatically when `loadWidgets()` runs and the `property` EAV table has a widget with that name for the current position. If the admin has not configured a widget named `hero_banner_widget` for the current page's position, the placeholder token will remain in the HTML unsubstituted (it will appear as literal text `{%hero_banner_widget%}` in the browser).

So manual composition requires coordination between the template author (who writes the placeholder) and the admin (who must create a widget instance with the matching name in the visual editor).

6.3 Hybrid Composition

A template can combine both modes: use `widgets-rows.tpl` for some positions (dynamic) and hardcode placeholders for others (manual). For example, the `landing_page_necotienda.tpl` template includes `widgets-common.tpl` (which uses `widgets-rows.tpl` for dynamic composition) but also appends hardcoded SVG sections after the widget-driven content:

```

1 <?php echo $header; ?>
2 <?php $tpl = is_dir(...) ? $this->config->get('config_template') : "choroni";
3   ?>
4 <div id="contentContainer" class="tpl-page" nt-editable>
5     <!-- Dynamic widget composition (via widgets-common.tpl) -->

```

```

6     <?php include(DIR_TEMPLATE . $tpl . "/shared/widgets-common.tpl"); ?>
7 </div>
8
9 <!-- Hardcoded content (not widget-driven) -->
10 <div class="svg-container-01" style="width: 2918px; height: 1306px;">
11     <?php include("landing_page_necotienda_svg_01.tpl"); ?>
12 </div>
13 <div class="svg-container-02" style="width: 2374px; height: 681px;">
14     <?php include("landing_page_necotienda_svg_02.tpl"); ?>
15 </div>

```

Listing 7: Hybrid composition in landing_page_necotienda.tpl.

This is the most flexible mode: the admin controls the main content area via the visual editor, and the template author controls the surrounding sections with hardcoded HTML.

7 The only: Position Prefix and Embedded Pages

7.1 The page_embed.tpl Layout

When a page is embedded (via `ControllerContentPage::embed()`, as called by the menu system's `submenu_type = 'page_id', v7`), the controller uses a simplified layout:

```

1 <?php $tpl = ...; ?>
2 <div class="tpl-page-embed">
3     <?php include(DIR_TEMPLATE . $tpl . "/shared/widgets-featured.tpl");?>
4
5     <div class="row">
6         <div class="large-12 medium-12 small-12">
7             <div nt-editable>
8                 <?php $position = 'main'; ?>
9                 <?php include(DIR_TEMPLATE . $tpl .
10                     "/shared/widgets-rows.tpl");?>
11             </div>
12         </div>
13     </div>
14
15     <?php include(DIR_TEMPLATE . $tpl .
16         "/shared/widgets-featured-footer.tpl");?>
17 </div>

```

Listing 8: page_embed.tpl — the simplified embed layout.

No side columns, no breadcrumbs — just the featured, main, and featured-footer positions, all in a single full-width column.

7.2 The only: Prefix

When embedding a page, the controller uses the `only:` position prefix:

```

1 $this->loadWidgets('only:featuredContent');
2 $this->loadWidgets('only:main');
3 $this->loadWidgets('only:featuredFooter');

```

Listing 9: The `only:` prefix in `ControllerContentPage::embed()`.

The `only:` prefix tells `Controller::loadWidgets()` to load *only* the per-object widgets (those matching the session's `object_type/object_id`), skipping the default widget tree. This is because an embedded page should only show its own widgets, not the entire page's widget tree.

In `Controller::loadWidgets()`, the `only:` prefix is detected at line 510:

```

1 if (!strpos($position, 'only:')) {
2     // Normal: load default tree + per-object tree
3     $rows = $widgets->getRows($params, ...);
4     // ... merge default rows ...
5 } else {
6     // 'only:' prefix: skip the default tree
7     // (the per-object query still runs below)
8 }
9 // Then, regardless of 'only:', the per-object query runs:
10 if ($session->has('object_type') || $session->has('object_id')) {
11     $params['position'] = $position = str_replace('only:', '', $position);
12     // ... query per-object widgets and merge ...
13 }

```

Listing 10: Controller::loadWidgets() — the only: detection.

So `only:` skips the first query (the default widget tree) and only runs the second query (the per-object widget tree). The result is that an embedded page shows only its own widgets, not the page's default widgets.

8 The CMS Lifecycle Trace

8.1 Visitor Views a Blog Post

1. Request: GET /articulos/my-first-post
2. SEO URL rewriter resolves my-first-post to post_id=42 via url_alias table.
3. Route: content/post.
4. ControllerContentPost::index():
 - (a) Sets session: object_type = 'post', object_id = 42, landing_page = 'content/post/index'.
 - (b) Loads post: modelPost->getById(42) — SELECT * FROM post p LEFT JOIN description pd ON (pd.object_id = p.post_id AND pd.object_type = 'post' AND pd.language_id = <lang>).
 - (c) Checks customer group permission via modelPost->getProperty(42, 'customer_groups', 'customer_groups').
 - (d) Loads widgets: loadWidgets('featuredContent'), loadWidgets('main'), loadWidgets('featuredFooter'). Each runs two queries: default tree + per-object tree (filtered by LIKE '%object_type=post%' and LIKE '%object_id=42%').
 - (e) Adds layout children: common/header, common/footer, common/column_left, common/column_right.
 - (f) Template resolution: checks EAV property(42, 'style', 'view') for a per-post template. If empty, uses config('default_view_post') or falls back to content/post.tpl.
 - (g) render(true):
 - Renders header, footer, column_left, column_right (each a child controller).
 - Renders each widget child (each a module controller).
 - Fetches content/post.tpl, which includes widgets-common.tpl, which includes widgets-rows.tpl, which emits {%widget_name%} tokens.
 - Substitutes tokens with widget HTML.
 - Post-processes HTML (strip comments, collapse whitespace).

- Returns the final HTML.
- (h) Cache write-back: `cache->set('html-post.42...', $html)`.
- (i) `response->setOutput($html)`.

8.2 Menu Embeds a Page

1. The “links” widget renders a menu link with `submenu_type = 'page_id', page_id = 7`.
2. `ControllerModuleLinks::getLinks()` calls `content/page->embed(7)`.
3. `ControllerContentPage::embed(7)`:
 - (a) Sets `session: object_type = 'page', object_id = 7, landing_page = 'content/page/index'`.
 - (b) Loads widgets with `only: prefix: loadWidgets('only:featuredContent'), loadWidgets('only:main'), loadWidgets('only:featuredFooter')`. This loads *only* the per-object widgets for page 7, skipping the default tree.
 - (c) Template: `content/page_embed.tpl` (simplified layout, no side columns).
 - (d) `render(true)` — renders the embedded page HTML.
 - (e) Returns the HTML to the links widget, which inserts it as the link’s submenu content.

9 Cross-Cutting Findings

9.1 Every CMS Page is a Widget Composition Surface

The most important finding of v8 is that every CMS page — whether a blog post, a static page, or a category listing — is a widget composition surface. The same `loadWidgets()/addChild()/render()` pipeline that drives the home page and product pages also drives CMS pages. This means the admin can configure different widgets for each individual post, page, or category — a “related posts” widget on one blog post, a “contact form” widget on a specific page, a “product list” widget on a category — all via the visual editor, without code changes.

9.2 The `widgets-common.tpl` Central Layout is a Refactoring

The `widgets-common.tpl` fragment was introduced to centralise the layout that was originally inlined in `home.tpl`. The home page template still uses the old inline pattern (manually including each fragment); newer templates (post, page, product, category) use the centralised `widgets-common.tpl`. This is an evolutionary refactoring scar — the home page was never migrated.

9.3 Manual Placeholders Require Admin Coordination

A template author can hardcode `{%widget_name%}` tokens in a custom template, but this requires the admin to create a widget instance with the matching name in the visual editor. If the admin doesn’t, the token appears as literal text in the browser. This coordination requirement makes manual composition fragile — a widget renamed or deleted in the admin breaks the template.

9.4 The `only:` Prefix is an Optimisation

The `only:` prefix halves the number of `NecoWidget::getRows()` queries for embedded pages (one query instead of two). This is important because embedded pages are rendered inside the links widget, which may render multiple embedded pages for a single menu’s submenu items. Without `only:`, each embedded page would fire two queries; with `only:`, it fires one.

9.5 Posts and Pages Share Everything But the Discriminator

The post and page models are nearly identical: same table (`post`), same primary key (`post_id`), same fields, same relations (`descriptions`, `stores`). The only difference is `$object_type` (`'post'` vs `'page'`) and the page model's `embed()` method. The `post_type` column in the database provides the discriminator. This is the same single-table-with-discriminator pattern used by many CMS platforms (WordPress's `post_type`, Drupal's `bundle`).

10 Closing Observations

10.1 The CMS + Widget Integration is the Platform's Crown Jewel

The integration of the CMS subsystem with the widget composition system is the platform's crown jewel. It allows a non-technical admin to create a blog post, assign widgets to it (a “related posts” carousel, a “comments” widget, a “social share” widget), choose a per-post template (a standard blog layout or a custom landing page), and publish — all without touching a single line of code. The widget composition system (v3) and the per-entity template override (v8) together form a page-builder experience that rivals dedicated CMS platforms.

10.2 The Manual Composition Mode is a Safety Valve

The manual composition mode (hardcoding `{%widget_name%}` tokens) is a safety valve for cases where the visual editor's layout capabilities are insufficient. A template author can create a custom template with a fixed HTML structure and insert widget placeholders at specific positions. This gives the admin control over the widget *content* (via the visual editor) while the template author controls the widget *layout* (via the hardcoded HTML).

10.3 Final Note

v8 confirms that the Necoyoad CMS subsystem is a widget-driven content management system where every page is a composition surface. The shared `post` table, the per-entity template override, the `widgets-common.tpl` universal layout, and the `{%widget_name%}` placeholder substitution together form a flexible page-builder architecture that supports both dynamic (admin-configured) and manual (hardcoded) composition modes.

A Glossary

Table 3: Glossary of CMS and widget composition terms.

Term	Meaning
<code>post</code> table	Shared by blog posts and static pages, discriminated by <code>post_type</code> .
<code>post_type</code>	The discriminator: <code>'post'</code> for blog posts, <code>'page'</code> for static pages.
Dynamic composition	Widgets configured via the visual editor, stored in the <code>property</code> EAV table, rendered via <code>widgets-rows.tpl</code> .
Manual composition	<code>{%widget_name%}</code> tokens hardcoded in a <code>.tpl</code> file. Requires matching widget instances in the admin.
Hybrid composition	A template that combines dynamic (via <code>widgets-common.tpl</code>) and hardcoded sections.
<code>widgets-common.tpl</code>	The universal layout fragment: featured + breadcrumbs + columns + featured-footer.
<code>widgets-rows.tpl</code>	The template that iterates the widget tree and emits <code>{%widget_name%}</code> tokens.
<code>only:</code> prefix	A position prefix that loads only per-object widgets, skipping the default tree. Used by embedded pages.
Per-entity template override	EAV <code>property('style', 'view')</code> returns a template path; falls back to config default; falls back to <code>choroni/</code> .
<code>page_embed.tpl</code>	Simplified layout for embedded pages (no side columns). Used by the menu system's <code>submenu_type = 'page_id'</code> .